TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

MÔN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ 24 HỆ THỐNG QUẢN LÝ CÔNG VIỆC TASK TRACKER

GIẢNG VIÊN HƯỚNG DẪN: HOÀNG VĂN TÂM

Sinh viên thực hiện: ĐẶNG QUỐC DOANH

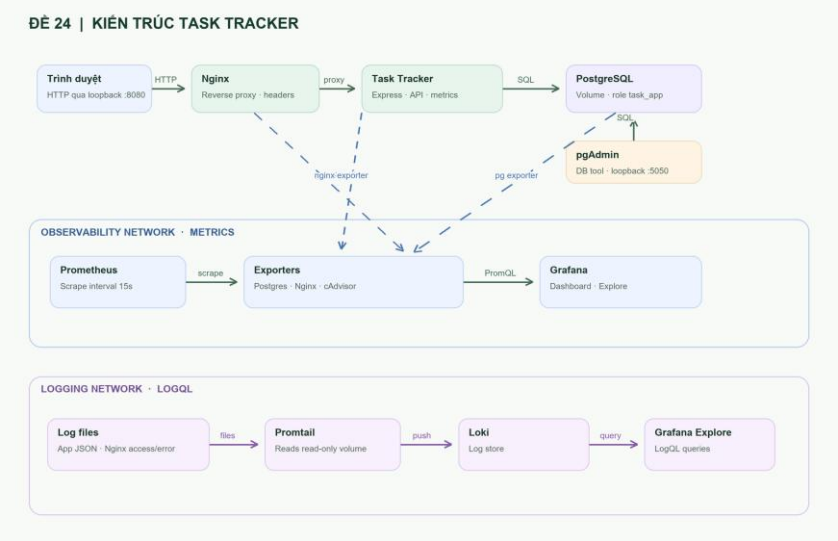
MSSV: DTC245200775

Lớp: KHMT K23A

THÁI NGUYÊN, NĂM 2026

1 Mục tiêu và thiết kế hệ thống

Đề tài triển khai ứng dụng quản lý công việc cá nhân bằng Node.js/Express, PostgreSQL và pgAdmin. Hệ thống hỗ trợ task, deadline và trạng thái; toàn bộ dịch vụ chạy bằng Docker Compose. Nginx cung cấp reverse proxy; Prometheus/Grafana giám sát; Loki/Promtail tập trung log. Báo cáo trình bày cấu hình, cách thao tác và kết quả đã ghi nhận kèm minh chứng.



Hình 1. Kiến trúc và các luồng HTTP, SQL, metrics, log

Luồng nghiệp vụ: trình duyệt → Nginx → Express → PostgreSQL. Prometheus scrape app và exporter; Grafana dùng PromQL để hiển thị. App/Nginx ghi file trên volume service_logs; Promtail đọc file, gửi Loki để Grafana Explore truy vấn LogQL.

Yêu cầu học phần	Nội dung và trang
GitHub và Docker Compose	Source, README, 3 mốc commit, khởi chạy — tr. 3
Task, deadline, trạng thái; PostgreSQL/pgAdmin	Chức năng giao diện, API và dữ liệu — tr. 4–5
Nginx reverse proxy	Proxy và 5 security headers — tr. 6
Prometheus và Grafana	Giám sát app, web, DB, container — tr. 7
Loki và Promtail	Log tập trung, 3 query LogQL — tr. 8
Hardening và nghiệm thu	Quyền, mạng, test và persistence — tr. 9–10

Phạm vi triển khai

Bản hiện tại phục vụ quản lý công việc cá nhân tại máy demo Windows. Chưa có đăng nhập hoặc phân quyền nhóm. HTTP kèm security headers đáp ứng lựa chọn của đề; không coi đây là triển khai HTTPS. Các ảnh là minh chứng các phiên đã chạy, không phải kết quả chạy mới của lần biên tập báo cáo này.

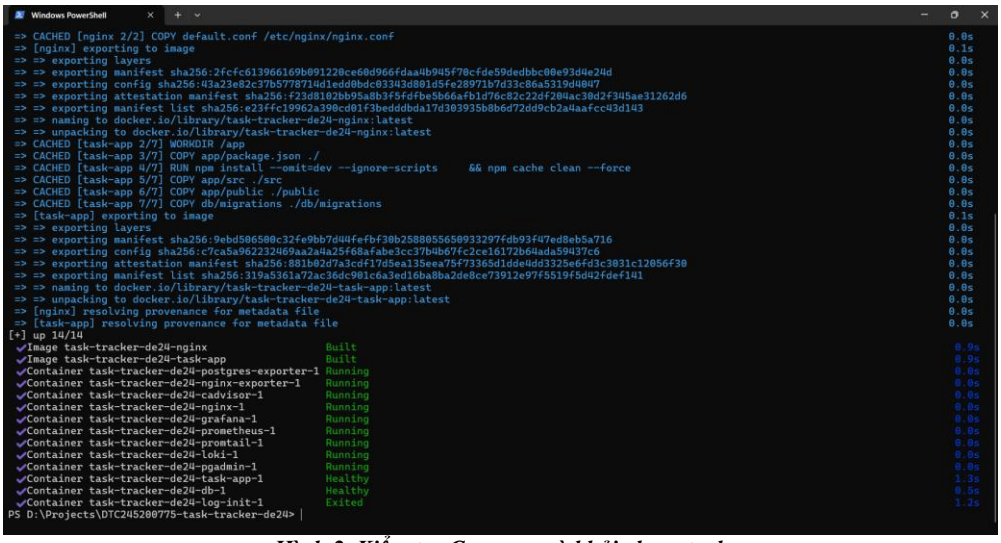
2 GitHub và triển khai bằng Docker Compose

Repository: github.com/quocdoanhdang8/DTC245200775-task-tracker-de24. Kho mã chứa ứng dụng, Dockerfile, Compose, cấu hình Nginx/monitoring, scripts kiểm tra và README hướng dẫn vận hành.

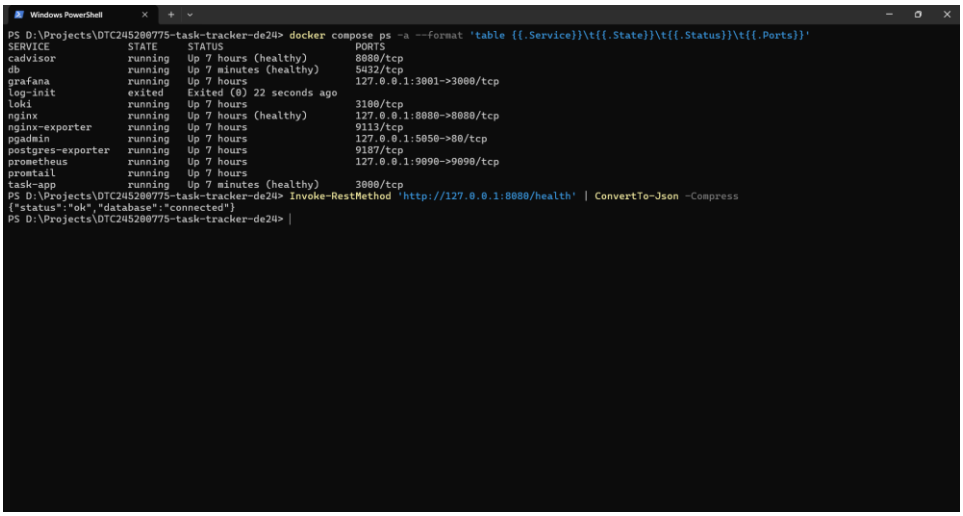
Commit	Nội dung triển khai
8d265c0	Ứng dụng, PostgreSQL, pgAdmin và Nginx
4c5451a	Prometheus, Grafana và exporters
fc35b1e	Loki, Promtail, hardening và báo cáo

Trên Windows, mở Docker Desktop ở chế độ Linux containers. Clone repository; với môi trường mới, sao chép .env.example thành .env và đặt mật khẩu riêng. Giữ .env ngoài Git; không ghi đè file môi trường đã có dữ liệu.

Tại thư mục dự án, chạy lần lượt: docker compose config --quiet, docker compose up -d --build, docker compose ps -a. Lệnh config --quiet kiểm tra cấu hình mà không in secrets sau nội suy.



Hình 2. Kiểm tra Compose và khởi chạy stack

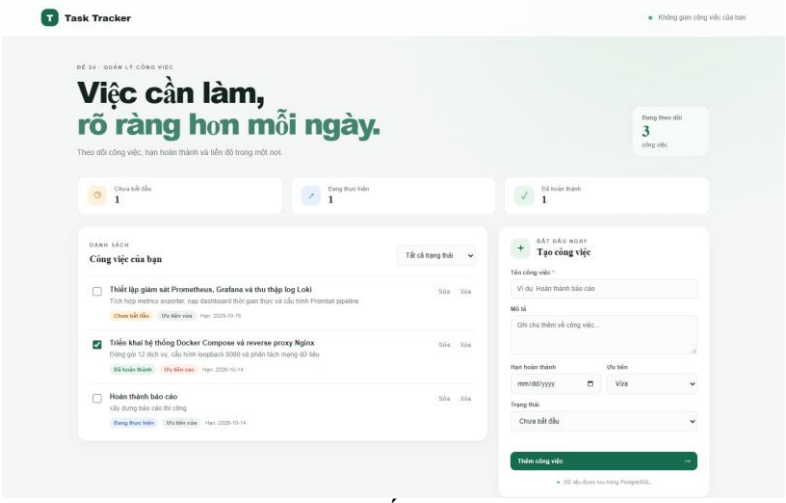


Hình 3. Danh sách service và port bindings sau khởi chạy

Có 12 service: 11 service vận hành và log-init chạy một lần để cấp quyền thư mục log rồi Exited (0). Trạng thái đó là hoàn thành thành công. Chỉ app, PostgreSQL, Nginx và cAdvisor có healthcheck trong cấu hình đã kiểm tra; không gọi mọi container là healthy.

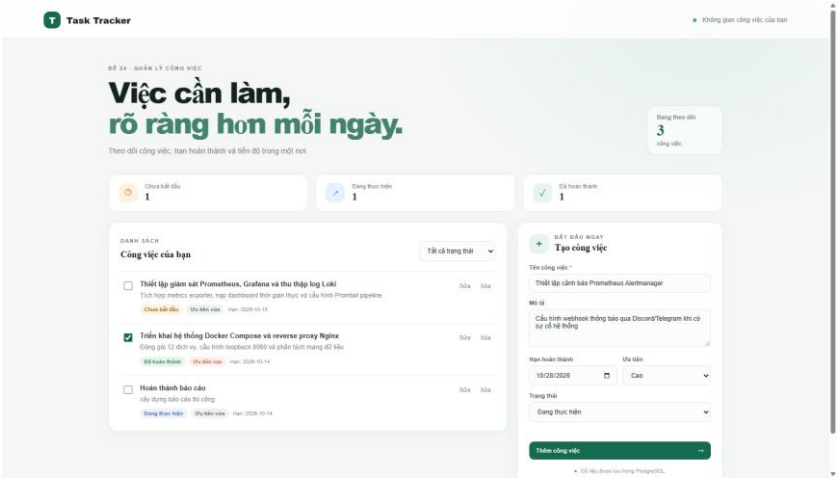
3 Chức năng quản lý công việc

Mở http://127.0.0.1:8080 qua Nginx. Ứng dụng cho phép tạo, xem, sửa, xóa task; tìm kiếm qua API và lọc trạng thái trên giao diện. Mỗi task có tiêu đề, mô tả, deadline, trạng thái và mức ưu tiên.



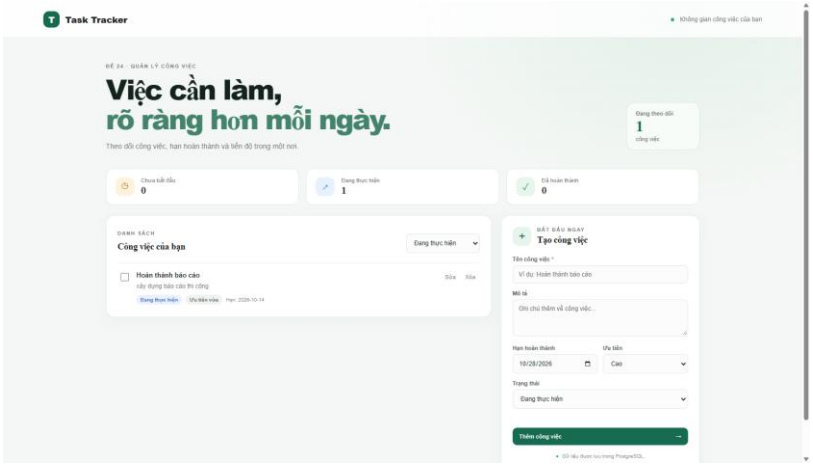
Hình 4. Danh sách công việc, thống kê trạng thái và form thao tác

Tạo task: nhập tiêu đề bắt buộc (tối đa 160 ký tự), mô tả (tối đa 4.000), ngày YYYY-MM-DD nếu có; chọn low/medium/high và todo/in_progress/done. Ngày bỏ trống được chuẩn hóa thành null; dữ liệu sai bị từ chối.



Hình 5. Nhập dữ liệu tạo công việc mới

Sửa task để cập nhật nội dung, deadline hoặc trạng thái; checkbox chuyển hoàn thành. Lọc "Đang thực hiện" chỉ hiển thị task in_progress. Khi demo, tạo một task riêng, sửa/lọc rồi xóa; không xóa task cần giữ làm minh chứng.

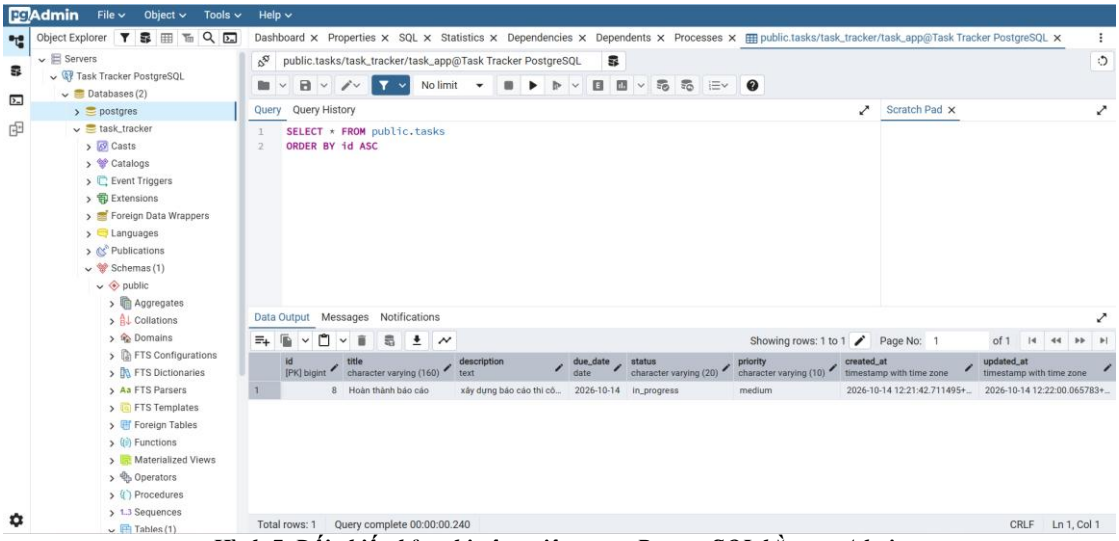


Hình 6. Lọc danh sách theo trạng thái đang thực hiện

4 PostgreSQL, pgAdmin và API

PostgreSQL lưu bảng public.tasks với id tự tăng, title, description, due_date (DATE), status, priority, created_at và updated_at. Ràng buộc đầu vào ở app và CHECK constraints ở DB hỗ trợ giữ dữ liệu hợp lệ; truy vấn nghiệp vụ dùng tham số hóa.

Mở pgAdmin tại http://127.0.0.1:5050. Kết nối host db, port 5432, database task_tracker, username task_app, mật khẩu TASK_DB_PASSWORD trong .env. Trong Query Tool, chạy: SELECT id, title, due_date, status, priority FROM public.tasks ORDER BY id;

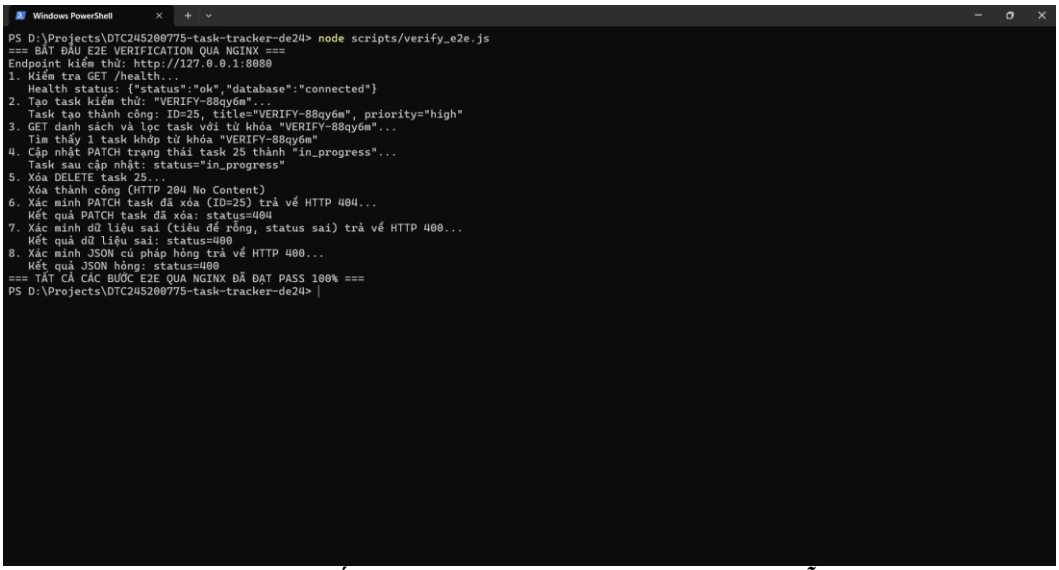


Hình 7. Đối chiếu bản ghi công việc trong PostgreSQL bằng pgAdmin

App chạy migration bằng role task_app; role này không phải superuser nhưng có USAGE, CREATE trên schema public. Quyền hiện tại rộng hơn CRUD. Exporter dùng role metrics_exporter và pg_monitor; PostgreSQL không publish cổng 5432 ra host.

Kiểm tra nghiệp vụ qua Nginx

Chạy node scripts/verify_e2e.js từ gốc repo. Kịch bản đi qua cổng 8080 và PostgreSQL thật: health 200; tạo 201; tìm lại 200; sửa 200; xóa 204; sửa task đã xóa 404; dữ liệu sai 400; JSON hỏng 400. Task thử được dọn, không cần xuất hiện trong ảnh pgAdmin.



Hình 8. Kết quả tám bước E2E tạo đọc sửa xóa và xử lý lỗi

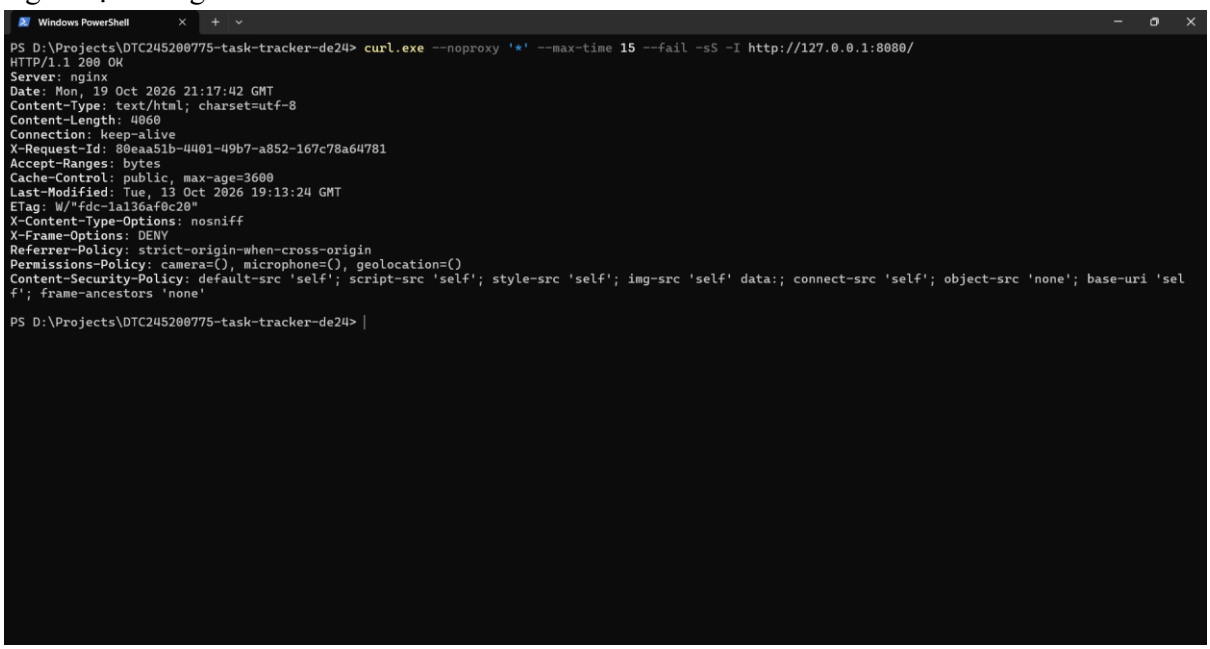
5 Nginx reverse proxy và bảo vệ HTTP

Nginx nhận request tại 127.0.0.1:8080 rồi proxy tới task-app:3000 trong mạng Docker; chuyển thông tin client qua headers. Người dùng truy cập website qua Nginx, trong khi cổng app 3000 không publish trực tiếp ra Windows.

Header	Tác dụng trong cấu hình
Content-Security-Policy	Giới hạn nguồn tải tài nguyên theo chính sách của trang
X-Frame-Options DENY	Ngăn nhúng trang trong frame
X-Content-Type-Options nosniff	Hạn chế trình duyệt đoán MIME type
Referrer-Policy	Kiểm soát thông tin referrer gửi sang trang khác
Permissions-Policy	Hạn chế camera, microphone và geolocation

Cách kiểm chứng

Trong PowerShell chạy curl.exe --noproxy '*' --max-time 15 -sS -I http://127.0.0.1:8080/, sau đó đọc mã HTTP và tìm đủ năm headers trong response. -I chỉ lấy headers; --noproxy bỏ proxy môi trường; --max-time giới hạn thời gian chờ.



Hình 9. Response qua Nginx có HTTP 200 và năm security headers

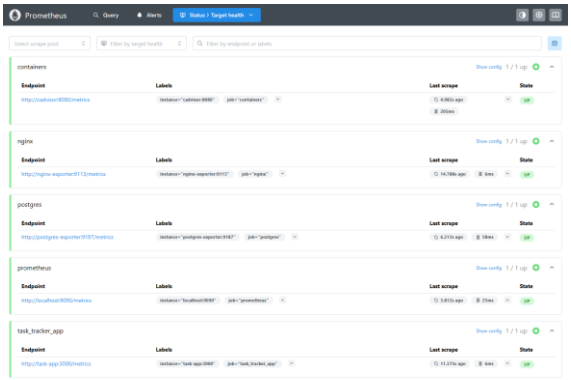
Ý nghĩa và giới hạn

Các headers hỗ trợ bảo vệ phía trình duyệt, không mã hóa dữ liệu trên đường truyền. Bản demo dùng HTTP kèm security headers theo lựa chọn đề cho phép. Nếu triển khai cho người dùng ngoài máy demo, cần bổ sung TLS, xác thực và rà soát quyền truy cập.

Nginx chạy non-root UID 101, read-only rootfs, bỏ toàn bộ capabilities và dùng no-new-privileges. Volume log có quyền ghi phù hợp; thư mục /tmp dùng tmpfs. Các biện pháp này được đối chiếu cùng app ở trang 9.

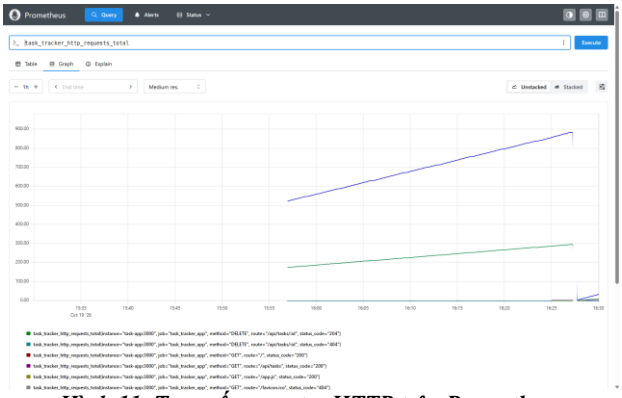
6 Giám sát bằng Prometheus và Grafana

Prometheus scrape mỗi 15 giây. Mở http://127.0.0.1:9090/targets: năm jobs gồm prometheus, task_tracker_app, postgres, nginx, containers. UP xác nhận scrape; pg_up=1 và nginx_up=1 xác nhận exporter đọc được dịch vụ tương ứng.



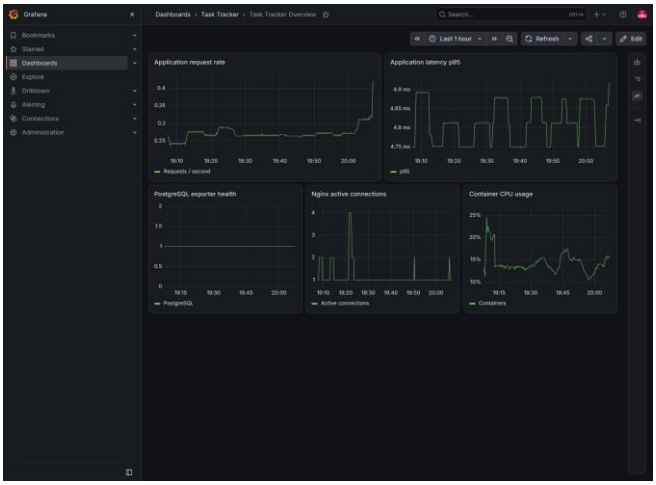
Hình 10. Năm targets ở trạng thái UP trong phiên kiểm tra

Phát sinh request rồi truy vấn task_tracker_http_requests_total. Đây là counter tổng request theo method, route, status_code; rate(...[5m]) biểu diễn tốc độ request theo giây.



Hình 11. Truy vấn counter HTTP trên Prometheus

Mở Grafana http://127.0.0.1:3001 → Task Tracker Overview; chọn time range có traffic rồi Refresh. Dashboard provision sẵn năm panel: request rate, latency p95, PostgreSQL health, Nginx connections và container CPU.



Hình 12. Dashboard giám sát ứng dụng web database và container

p95 ước tính từ histogram; khi No data cần kiểm tra targets, datasource và time range. Số liệu cAdvisor phụ thuộc lớp máy ảo Linux của Docker Desktop.

7 Log tập trung với Loki và Promtail

App ghi JSON tại /var/log/app/app.jsonl; Nginx ghi access/error trong /var/log/nginx. Volume service_logs mount vào /var/log ở app/Nginx, và chỉ đọc tại /var/log/collected ở Promtail. Promtail thêm labels, gửi Loki; Grafana Explore truy vấn qua datasource Loki.

Ba truy vấn LogQL

```
{service="task-tracker"} | json
```

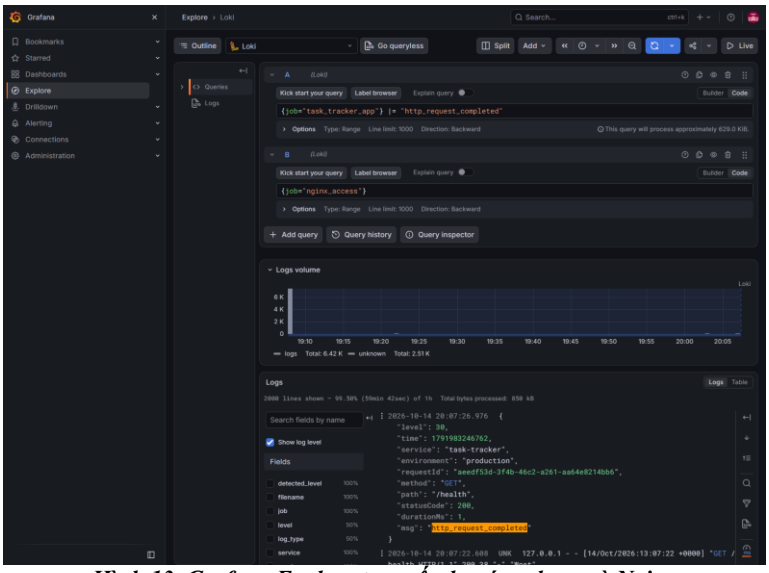
Đọc log ứng dụng và phân tích các trường method, path, statusCode, requestId, durationMs.

```
{service="task-tracker"} | json | statusCode >= 200 and statusCode < 300
```

Lọc request thành công 2xx; chỉ dùng >= 200 sẽ lấy cả lỗi 4xx và 5xx.

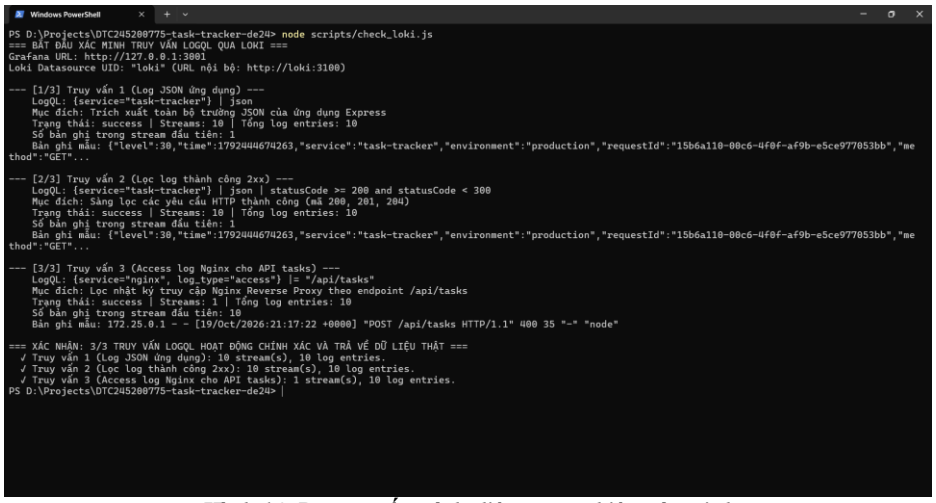
```
{service="nginx", log_type="access"} |= "/api/tasks"
```

Tìm access log chứa đường dẫn API công việc; đối chiếu method, mã HTTP và thời gian.



Hình 13. Grafana Explore truy vấn log ứng dụng và Nginx

Chạy node scripts/check_loki.js để kiểm chứng qua API. Script chỉ báo thành công khi cả ba query có log entries; nếu chưa có dữ liệu, tạo GET an toàn rồi retry có giới hạn. Vẫn rỗng hoặc lỗi thì exit 1.



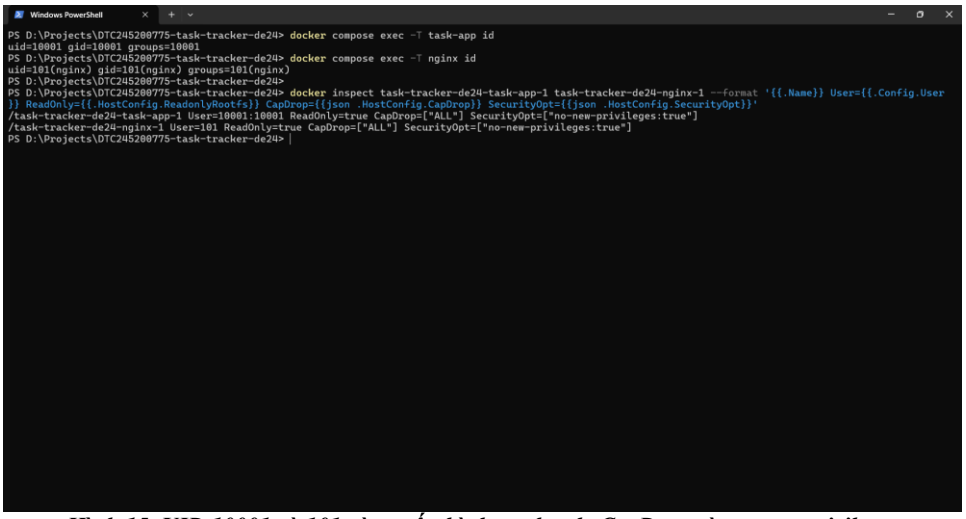
Hình 14. Ba truy vấn có dữ liệu trong phiên xác minh

Lỗi database giả lập trong unit test không mặc nhiên là log runtime Loki. Các dòng HTTP 400 từ E2E có thể là ca âm chủ động, cần đọc ngữ cảnh trước khi kết luận hệ thống lỗi.

8 Hardening mạng và quyền truy cập

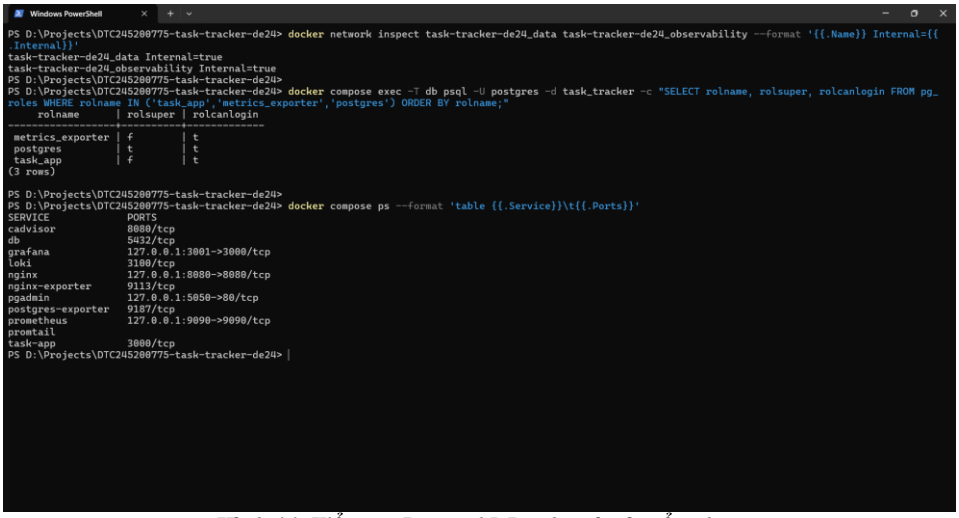
Các biện pháp được áp dụng theo phạm vi từng service, kết hợp giảm quyền tiến trình, hạn chế filesystem, phân tách mạng và bảo vệ secrets. Không khẳng định tất cả container đều non-root hoặc có cùng hardening.

Chạy docker compose exec -T task-app id và docker compose exec -T nginx id. Inspect app/Nginx để đối chiếu User, ReadonlyRootfs, CapDrop và SecurityOpt.



Hình 15. UID 10001 và 101 cùng cấu hình read-only CapDrop và no-new-privileges

Biện pháp	Phạm vi và cách hoạt động
Non-root và giảm quyền	App UID 10001, Nginx UID 101; cap_drop ALL; no-new-privileges
Filesystem chỉ đọc	App/Nginx read_only; vẫn ghi vào volume log và tmpfs /tmp
Network isolation	data, observability là internal; front, management là bridge thường
Cổng và tài khoản	UI bind 127.0.0.1; app/DB không publish 3000/5432; role DB không superuser
Secrets và SQL	Mật khẩu riêng trong .env được ignore; SQL nghiệp vụ tham số hóa

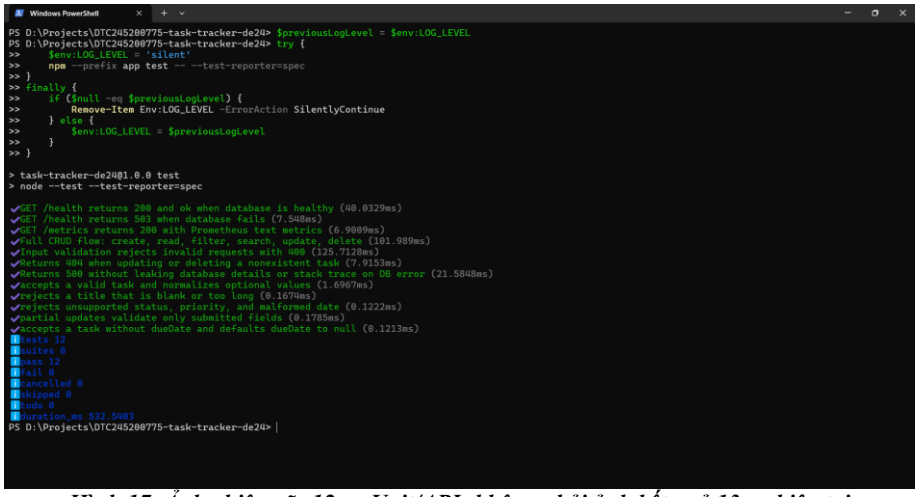


Hình 16. Kiểm tra Internal DB roles và các cổng host

DB chỉ nối data và không publish cổng; không mô tả là "cô lập hoàn toàn khỏi host". Container đa mạng có thể đi ra qua front/management. log-init chạy root một lần để chown /logs/app và /logs/nginx rồi exit 0. Có thể tách role migration để thu hẹp quyền task_app khi vận hành lâu dài.

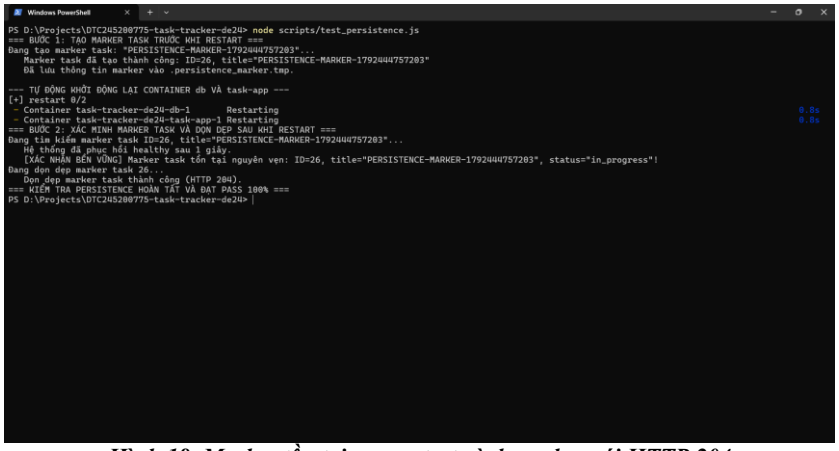
9 Kiểm thử tổng kết và vận hành

Ảnh Hình 17 ghi kết quả phiên cũ: 12 ca Unit/API dùng database giả lập. Code hiện tại đạt 13/13 Unit/API và QA harness 20/20 trong phiên kiểm tra trước; E2E trang 5 là kiểm tra riêng với PostgreSQL thật.



Hình 17. Ảnh phiên cũ: 12 ca Unit/API; không phải ảnh kết quả 13 ca hiện tại

Chạy node scripts/test_persistence.js: tạo marker, restart db/task-app, chờ health, đối chiếu task còn nguyên rồi xóa. Nếu còn marker dở, chạy scripts/persistence_step2.js để xác minh/dọn. Đây là kiểm tra restart đã thực hiện, không thay cho backup/restore.



Hình 18. Marker tồn tại sau restart và được dọn với HTTP 204

Runtime Windows do người dùng cung cấp: Compose 11 service, log-init exit 0; E2E 8/8; Loki 3/3; persistence marker ID 29 còn qua restart rồi được dọn; Prometheus 5/5 UP, pg_up=1, nginx_up=1; Grafana có dữ liệu ở 5 panel.

Vận hành và trình tự demo

URL: website :8080; pgAdmin :5050; Prometheus :9090; Grafana :3001, đều tại 127.0.0.1. Demo: Compose → tạo/sửa/lọc task → pgAdmin → headers → targets/dashboard → ba LogQL → UID/mạng/role → persistence. Dừng bằng docker compose down để giữ volumes; down -v xóa dữ liệu. Đổi .env không tự đổi mật khẩu tài khoản trong volumes.

Kết luận và tài liệu tham khảo

Hệ thống đáp ứng phạm vi quản lý công việc cá nhân và các thành phần triển khai của Đề 24. Hướng phát triển: xác thực nhóm, TLS, backup/restore và cảnh báo. Promtail được giữ theo yêu cầu học phần; cần đánh giá công cụ thay thế khi nâng cấp.

Tài liệu: Docker Compose; PostgreSQL 17; Express; Nginx; Prometheus; Grafana; Loki. Mã nguồn và hướng dẫn chạy: repository tại trang 3.